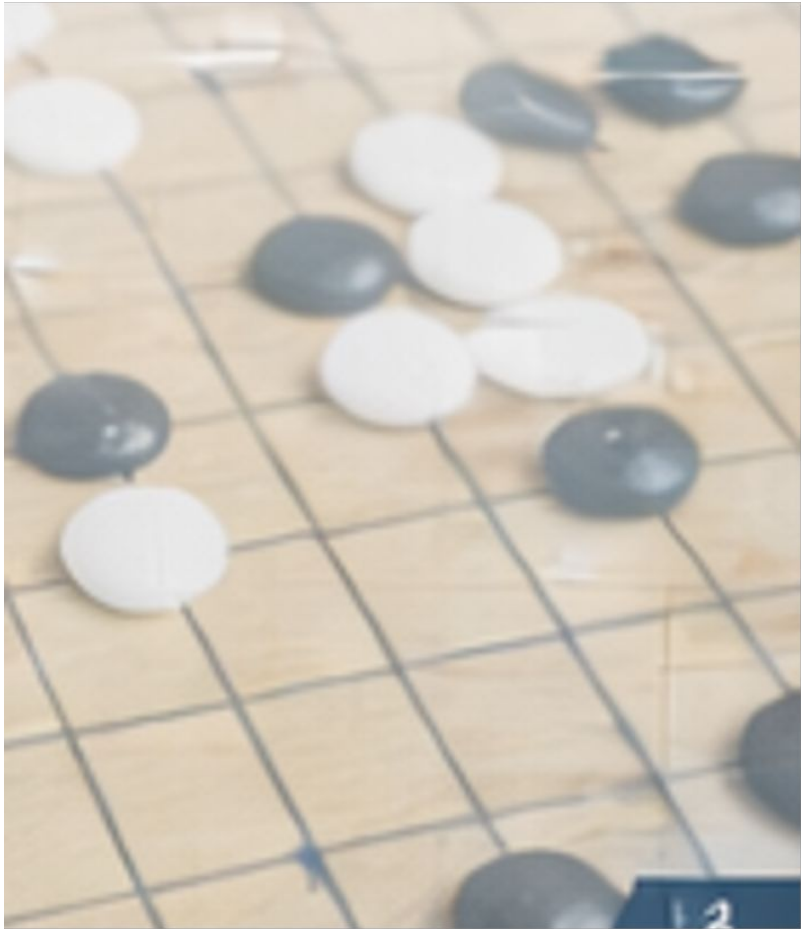
Making Self-Play Reinforcement Learning Feasible for Gomoku via Candidate Move Restriction

Zengzheng Jiang • Peter Wei • Weizhi Du



Problem Statement

Can an interpretable pruning rule make AlphaZero-style self-play practical on a larger Gomoku board?

- End-to-end 6x6 AlphaZero pipeline works.
- The framework was migrated to 8x8, which makes the state-action space much larger (48 hours training).
- Extending the same setup to the standard 15x15 board for human play would incur a prohibitive training cost.
- The reported bottleneck is MCTS, so pruning is the most direct place to save computations.

Reduce the effective action space so self-play converges faster without giving up much playing strength.

48h

Unrestricted 8x8 baseline training time

Interpretation

When the board gets bigger, many actions are irrelevant. If we can keep only local or high-value moves before expansion, MCTS should search a much smaller tree.

Pipeline Construction

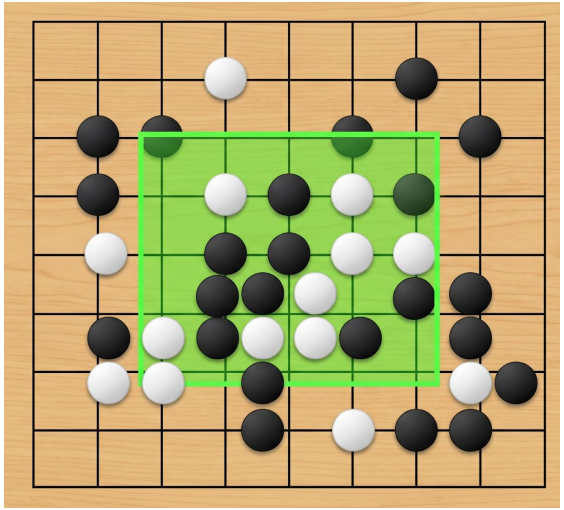
- **train.py** orchestrates self-play, learning, evaluation, and checkpointing
- **game.py** constructs board state, legal moves, win checking, full self-play game
- **policy_value_net_pytorch.py** outputs move probabilities and state value
- **mcts_alphaZero_pruned.py** performs playout, NN evaluation, pruning-aware child expansion
- **move_pruning.py** computes local candidate window and keeps legal moves only

Training loop

Self-play (states, policy, labels)
↓
Replay buffer + augmentation
↓
Policy-value network updates
↓
Evaluation against pure MCTS
↓
Save current / best model

pruning is ready for timing + strength comparison against the best saved 8x8x5 policy.

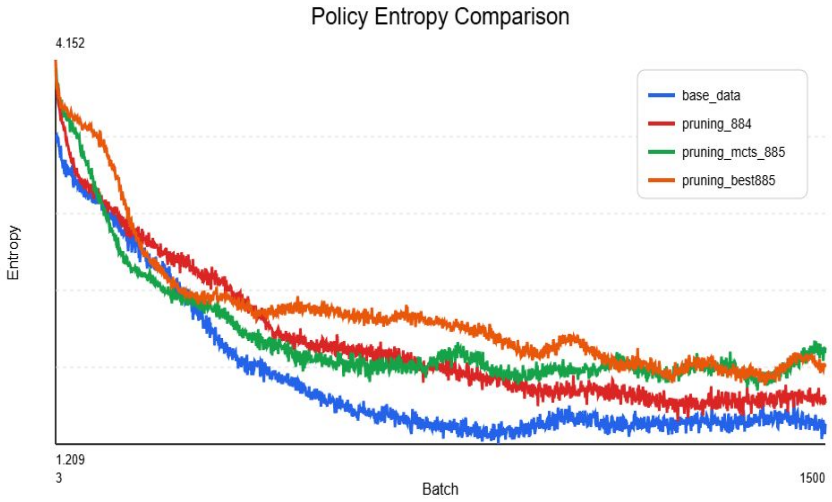
Where pruning enters the move decision



Inside one MCTS playout

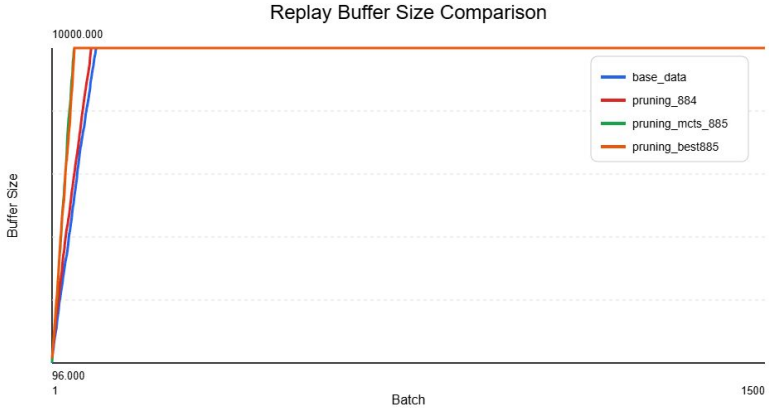
1. Select from the root down to a leaf node.
2. Call policy_value_fn(state) to get action_probs and leaf_value.
3. Call get_pruned_moves(state, window_size = 6) and keep only legal moves in that window.
4. Filter action_probs, expand only those children, then backpropagate the result.

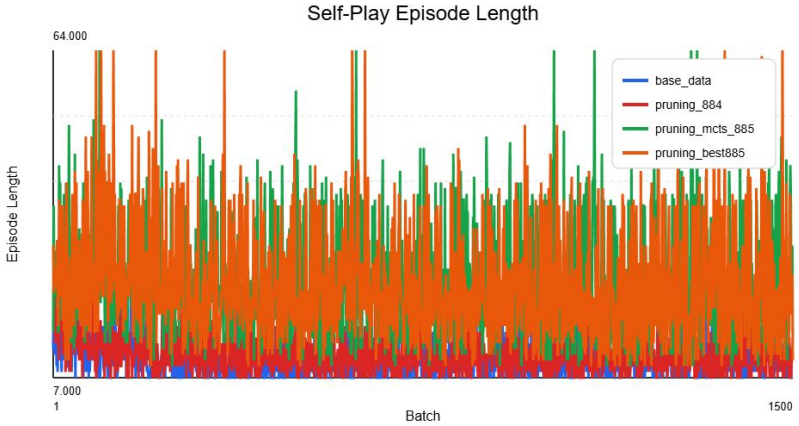
Training curves: pruning learns meaningful policies

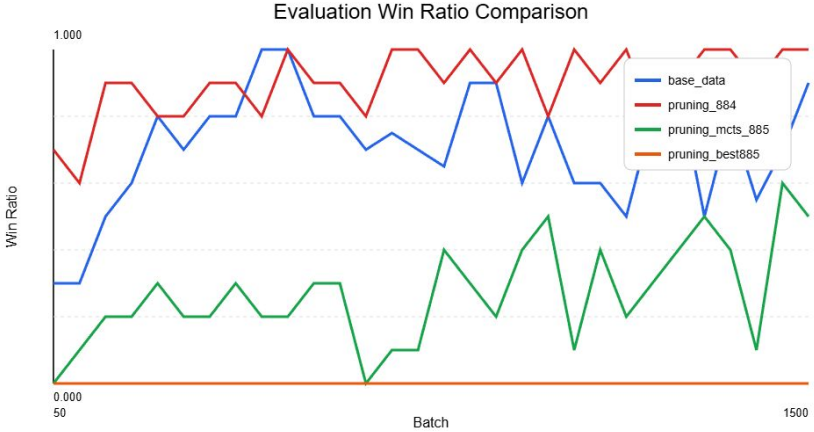


- Training loss goes down and policy entropy drops, learning structured policies.
- 6x6 baseline is easier to optimize. 8×8 pruned runs sit higher due to harder board and win condition.
- The runtime gain is not coming from skipping learning. We still see real training dynamics with more feasible budget.

Replay buffer size, Self-play episode length, Win ratios







- Replay buffer fills quickly and the training loop is stable.
- 8x8 connect-4 is very strong. Connect-4 is less stable.
- Pruning solves a compute bottleneck, but we still need strength preservation on the hardest settings.

Ablation settings and runtime comparison

Model	Board/Rule	Running Time	Win Ratio
Original Baseline	6×6 board, connect-4	3 hours	0.89
Original Baseline	8×8 board, connect-5	48 hours	na
Pruned Model	8×8 board, local 6×6 window, connect-4	3 hours	1.00
Pruned Model (MCTS)	8×8 board, local 6×6 window, connect-5	7 hours	0.50

*Experiments were run on an NVIDIA GeForce RTX 3070 Laptop GPU

- Scaling from 6x6 connect-4 to 8x8 connect-5 increases runtime from 3 hours to 48 hours.
- On the same 8×8 connect-5 task, local pruning reduces runtime from 48 hours to 7 hours (6.9x).
- Pruning also makes the larger board run (8x8 connect-4) in the same 3-hour range as the 6x6 baseline.

Significance and next steps

- **Completed work:** the AlphaZero-style Gomoku pipeline, pruning-enabled MCTS, self-play training, evaluation, and checkpointing are all running end-to-end.
- **Empirical result:** local candidate restriction reduces runtime while preserving capacity, so larger-board self-play becomes much more practical.
- The hardest setting still shows a trade-off between efficiency and strength. The pruning method saves time costs for researchers, making it easier to tune parameters.

Next steps:

- **Pruning method:** local-window pruning plus threat-aware exceptions or policy top-k filtering.
- **Ablation study:** compare different levels of pruning, and report runtime, candidate move count, and final win ratio.
- **Trade-off curve:** computation saved vs. playing strength preserved.

Thank you!